

# Sistema Web Gaby Salón y Spa

Documentación técnica completa

Guía explicada paso a paso — desde cómo instalar y usar el programa, hasta cómo funciona cada módulo por dentro. Pensada también como material de estudio para presentar el proyecto.

Sitio informativo + panel de administración

Tecnología: Next.js · React · Firebase

# Índice

---

1. ¿Qué es este sistema?
2. Palabras técnicas explicadas (glosario rápido)
3. Tecnologías utilizadas
4. Cómo instalar y correr el programa (desde cero)
5. Configuración de Firebase (paso a paso)
6. Cómo usar el sistema
7. Organización de las carpetas del proyecto
8. La base de datos (Firebase / Firestore)
9. Módulo 1 — Sitio público (la página que ven las clientas)
10. Módulo 2 — Inicio de sesión de administradores
11. Módulo 3 — Servicios
12. Módulo 4 — Reservas
13. Módulo 5 — Registro diario
14. Módulo 6 — Reportes
15. Flujo completo del sistema (ejemplo de principio a fin)
16. Seguridad (reglas de la base de datos)
17. Preguntas típicas de la exposición (con respuestas)
18. Problemas comunes y soluciones

# 1. ¿Qué es este sistema?

---

Es una **página web** para el salón de belleza **Gaby Salón y Spa**. Tiene dos partes:

- **La parte pública:** lo que ve cualquier persona que entra a la página. Muestra la información del salón (servicios, precios, galería de fotos, contacto) y permite **reservar una cita** por internet.
- **La parte de administración (panel):** una zona privada, protegida con usuario y contraseña, donde el salón administra sus servicios, ve las reservas de las clientas, anota lo que se hace cada día y consulta reportes de ingresos.

**En pocas palabras:** las clientas usan la parte pública para informarse y reservar; el salón usa el panel privado para llevar el control de su negocio.

## ¿Para qué sirve tener todo esto conectado?

La gracia del sistema es que las partes están **enlazadas**: los servicios que carga el salón aparecen en la página y en el formulario de reservas; las reservas y los servicios realizados alimentan los reportes de ingresos. Todo trabaja junto y la información se guarda en la nube (internet), no en un cuaderno.

## 2. Palabras técnicas explicadas

---

Antes de seguir, acá tenés el significado de las palabras que se repiten en este documento, explicadas de forma sencilla:

| Palabra                     | Qué significa (fácil)                                                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Código</b>               | Las instrucciones escritas que le dicen a la computadora qué hacer.                                                       |
| <b>Servidor</b>             | Un programa que "sirve" la página web para que se pueda ver en el navegador.                                              |
| <b>Navegador</b>            | El programa donde se ve internet (Chrome, Edge, Safari...).                                                               |
| <b>Base de datos</b>        | Un lugar en internet donde se guarda la información (servicios, reservas, etc.), como un gran archivador digital.         |
| <b>Firebase / Firestore</b> | El servicio de Google que usamos como base de datos y para el inicio de sesión.                                           |
| <b>Módulo</b>               | Una parte del sistema que cumple una función específica (ej: el módulo de reservas).                                      |
| <b>Componente</b>           | Una pieza reutilizable de la página (ej: el menú de arriba, un botón, una tarjeta).                                       |
| <b>Colección</b>            | Un grupo de datos del mismo tipo dentro de la base de datos (ej: la colección de "servicios").                            |
| <b>Documento</b>            | Una ficha individual dentro de una colección (ej: un servicio en particular).                                             |
| <b>Autenticación</b>        | El proceso de comprobar que alguien es quien dice ser (el inicio de sesión).                                              |
| <b>Localhost</b>            | La dirección para ver la página en tu propia computadora mientras la desarrollás:<br><code>http://localhost:3000</code> . |

### 3. Tecnologías utilizadas

---

El sistema está construido con herramientas modernas y gratuitas. Aquí van explicadas:

| Tecnología              | Para qué sirve                                                                                               |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| Next.js                 | El "esqueleto" del proyecto. Organiza las páginas y hace que la web funcione rápido.                         |
| React                   | La herramienta con la que se construyen las pantallas y los botones que reaccionan a lo que hace el usuario. |
| TypeScript              | El lenguaje de programación en el que está escrito el código (una versión ordenada de JavaScript).           |
| Tailwind CSS            | La herramienta para darle diseño, colores y estilo a la página.                                              |
| Firebase Authentication | Maneja el inicio de sesión seguro de los administradores.                                                    |
| Cloud Firestore         | La base de datos donde se guardan servicios, reservas y registros.                                           |
| Framer Motion           | Le da las animaciones suaves a la página (transiciones, aparecidos).                                         |
| Lucide Icons            | La librería de iconos (candado, tijera, calendario, etc.).                                                   |

**¿Por qué estas y no otras?** Son gratuitas, muy usadas en la industria y funcionan juntas sin complicaciones. Firebase evita tener que programar y pagar un servidor propio, lo cual simplifica mucho el proyecto.

## 4. Cómo instalar y correr el programa (desde cero)

---

Estos pasos son para alguien que **nunca** ha corrido un programa. Seguílos en orden.

### Paso 1 — Instalar Node.js

Node.js es el programa que permite ejecutar el proyecto. Sin él, nada funciona.

1. Entrá a <https://nodejs.org>
2. Descargá la versión que dice **"LTS"** (la recomendada).
3. Abrí el archivo descargado y seguí la instalación con "Siguiente / Continuar" hasta el final.

### Paso 2 — Descargar el proyecto

El proyecto está guardado en internet (GitHub). Hay que traerlo a la computadora.

**Opción fácil:** entrá a <https://github.com/JCortes23/GabySalaon>, hacé clic en el botón verde **"Code"** y luego en **"Download ZIP"**. Descomprimí (extraé) la carpeta en un lugar fácil de encontrar, por ejemplo el Escritorio.

### Paso 3 — Abrir la carpeta en la "Terminal"

La **Terminal** es una ventana donde se escriben comandos. En Mac se llama "Terminal", en Windows "Símbolo del sistema" o "PowerShell".

1. Abrí la Terminal.
2. Escribí `cd` (con un espacio después) y arrastrá la carpeta del proyecto a la ventana; se pega la ruta sola. Presioná **Enter**.

Esto le dice a la Terminal: "trabajá dentro de esta carpeta".

### Paso 4 — Instalar las piezas que necesita el proyecto

Escribí este comando y presioná Enter:

```
npm install
```

Va a tardar 1–2 minutos descargando todo lo necesario. Es normal ver muchos mensajes.

## Paso 5 — Conectar la base de datos

El proyecto necesita unas "llaves" secretas para conectarse a Firebase. Se guardan en un archivo llamado `.env.local`. Cómo obtener esas llaves se explica en el **capítulo 5**.

1. En la carpeta del proyecto hay un archivo de ejemplo: `.env.local.example`.
2. Hacé una copia y renombrala como `.env.local`.
3. Adentro, pegá las llaves de tu proyecto de Firebase.

**Importante:** el archivo `.env.local` contiene datos privados y por eso **nunca se sube a internet**. Cada computadora tiene el suyo.

## Paso 6 — Correr el programa

Escribí este comando y presioná Enter:

```
npm run dev
```

Cuando aparezca el mensaje **"Ready"**, el programa ya está corriendo.

## Paso 7 — Ver la página

Abrí el navegador y entrá a: `http://localhost:3000`

Para **apagar** el programa, volvé a la Terminal y presioná **Ctrl + C**. Para volver a encenderlo, repetí el Paso 6.

## 5. Configuración de Firebase (paso a paso)

---

Firebase es el servicio de Google que usa el proyecto para dos cosas: el **inicio de sesión** (Authentication) y la **base de datos** (Firestore). Hay que configurarlo una sola vez. Todo es gratuito.

### Paso 1 — Crear el proyecto

1. Entrá a `https://console.firebase.google.com` e iniciá sesión con una cuenta de Google.
2. Clic en "**Crear un proyecto**", ponedle un nombre (ej: `gabysalón`) y aceptá.
3. Podés desactivar Google Analytics (no hace falta). Clic en "Crear proyecto".

### Paso 2 — Registrar la aplicación web y copiar las llaves

1. En la pantalla principal del proyecto, clic en el ícono web `</>`.
2. Poné un apodo y registrá la app.
3. Firebase muestra un bloque llamado `firebaseConfig` con varias llaves (`apiKey`, `projectId`, etc.). **Esas llaves** son las que van en el archivo `.env.local` (ver capítulo 4, Paso 5).

### Paso 3 — Activar el inicio de sesión

1. En el menú, entrá a **Authentication** → **Comenzar**.
2. En "Sign-in method", activá "**Correo electrónico/contraseña**" y guardá.
3. En la pestaña **Users**, creá un usuario administrador con su correo y contraseña. **Ese** es el usuario para entrar al panel.

### Paso 4 — Crear la base de datos

1. En el menú, entrá a **Firestore Database** → **Crear base de datos**.
2. Elegí una ubicación cercana (ej: `nam5 / us-central`) — **no se puede cambiar después**.
3. Elegí "Iniciar en modo de producción" y creá.

### Paso 5 — Publicar las reglas de seguridad



1. En Firestore, entrá a la pestaña **Reglas (Rules)**.
2. Copiá el contenido del archivo `firestore.rules` del proyecto y pegalo ahí.
3. Clic en **Publicar**. (Qué hacen esas reglas se explica en el capítulo 16.)

## Paso 6 — Cargar los servicios iniciales

Iniciá sesión en `/admin`, entrá al módulo **Servicios** y hacé clic en "**Importar servicios actuales**". Esto copia los servicios de ejemplo a la base de datos de una sola vez.

## 6. Cómo usar el sistema

---

### Como clienta (parte pública)

1. Entrá a <http://localhost:3000>.
2. Se ve la información del salón: servicios con precios, galería, testimonios y contacto.
3. Para reservar, hacé clic en "**Reservar**" (arriba) o entrá a <http://localhost:3000/reservar>.
4. Elegí el servicio, la fecha, la hora, escribí tu nombre y WhatsApp, y confirmá.

### Como administrador (panel privado)

1. Entrá a <http://localhost:3000/admin> (o hacé clic en el candado del menú).
2. Iniciá sesión con el correo y contraseña creados en Firebase.
3. Vas a ver 4 módulos: **Servicios**, **Reservas**, **Registro diario** y **Reportes**.

## 7. Organización de las carpetas del proyecto

---

El proyecto está ordenado en carpetas. Cada una tiene una función:

| Carpeta / archivo                     | Qué contiene                                                                           |
|---------------------------------------|----------------------------------------------------------------------------------------|
| <code>src/app/</code>                 | Las páginas del sistema (inicio, reservar, y el panel admin).                          |
| <code>src/app/admin/</code>           | Todas las pantallas privadas del administrador.                                        |
| <code>src/components/</code>          | Piezas reutilizables (menú, pie de página, tarjetas, botón de WhatsApp, etc.).         |
| <code>src/components/sections/</code> | Las secciones de la página principal (inicio, servicios, galería...).                  |
| <code>src/lib/</code>                 | El "cerebro" que se comunica con la base de datos (Firestore).                         |
| <code>src/content/site.ts</code>      | La información del salón (nombre, contacto, horario, servicios).<br>Todo se edita acá. |
| <code>public/images/</code>           | Las imágenes y fotos que se muestran.                                                  |
| <code>firestore.rules</code>          | Las reglas de seguridad de la base de datos.                                           |
| <code>.env.local</code>               | Las llaves privadas de conexión a Firebase.                                            |

## 8. La base de datos (Firestore / Firebase)

La información se guarda en **Cloud Firestore**, la base de datos de Google. Firestore organiza los datos en **colecciones** (grupos) y cada dato es un **documento** (una ficha con campos).

Este sistema usa **4 colecciones**:

### Colección **services** (servicios)

| Campo    | Qué guarda                                                   |
|----------|--------------------------------------------------------------|
| category | La categoría (Cabello, Uñas, Spa & Estética).                |
| name     | El nombre del servicio.                                      |
| desc     | La descripción.                                              |
| price    | El precio en colones.                                        |
| order    | El orden en que aparece.                                     |
| active   | Si está activo ( <b>true</b> ) o archivado ( <b>false</b> ). |

### Colección **reservations** (reservas)

| Campo                           | Qué guarda                                                             |
|---------------------------------|------------------------------------------------------------------------|
| date / time                     | La fecha y hora de la cita.                                            |
| serviceName / category / price  | Copia del servicio elegido (para no perder el dato si cambia después). |
| customerName /<br>customerPhone | Nombre y WhatsApp de la clienta.                                       |
| status                          | Estado: pendiente, confirmada, completada o cancelada.                 |
| slotId                          | Identificador de la franja horaria (para controlar los cupos).         |

### Colección **slots** (cupos por hora)

| Campo       | Qué guarda      |
|-------------|-----------------|
| date / time | La hora exacta. |

|       |                                                                     |
|-------|---------------------------------------------------------------------|
| count | Cuántas reservas ocupan esa hora (para no pasarse de la capacidad). |
|-------|---------------------------------------------------------------------|

**Colección** **records** **(registro diario)**

| Campo                          | Qué guarda                            |
|--------------------------------|---------------------------------------|
| date                           | El día en que se realizó el servicio. |
| serviceName / category / price | Copia del servicio realizado.         |
| quantity                       | Cuántas veces se hizo.                |
| note                           | Una nota opcional.                    |

## 9. Módulo 1 — Sitio público

Archivos `src/app/page.tsx` y `src/components/sections/`

Es la página principal que ve cualquier persona. Está formada por secciones que se muestran una debajo de otra:

| Sección       | Qué muestra                                                 | Archivo          |
|---------------|-------------------------------------------------------------|------------------|
| Menú superior | Logo y botones de navegación, Reservar y WhatsApp.          | navbar.tsx       |
| Inicio (Hero) | La portada con el eslogan y botones principales.            | hero.tsx         |
| Quiénes somos | Presentación del salón.                                     | about.tsx        |
| Servicios     | Lista de servicios con precios (viene de la base de datos). | services.tsx     |
| Galería       | Fotos de trabajos, con vista ampliada.                      | gallery.tsx      |
| Testimonios   | Opiniones de clientes (carrusel).                           | testimonials.tsx |
| Contacto      | Dirección, horarios, mapa y redes.                          | contact.tsx      |

### Detalle técnico importante

La sección de **Servicios** se conecta a la base de datos y muestra solo los servicios **activos**. Si por algún motivo no logra conectarse, muestra una lista de respaldo que está escrita en el archivo `site.ts`, para que la página nunca se vea vacía.

## 10. Módulo 2 — Inicio de sesión de administradores

---

**Archivos** `src/app/admin/login/page.tsx` · `src/lib/firebase.ts`

Protege el panel privado. Solo quien tenga usuario y contraseña puede entrar.

### Cómo funciona

1. La persona escribe su correo y contraseña.
2. El sistema le pregunta a **Firebase Authentication** si esos datos son correctos.
3. Si son correctos, la lleva al panel. Si no, muestra un mensaje de error.
4. Todas las pantallas del panel verifican que haya una sesión activa; si no, devuelven al inicio de sesión automáticamente.

Las contraseñas **no se guardan en el código** ni en la base de datos del proyecto: las administra Firebase de forma segura.

## 11. Módulo 3 — Servicios

---

**Archivos** `src/app/admin/services/page.tsx` · `src/lib/services.ts`

Permite administrar la lista de servicios del salón.

### Qué se puede hacer

- **Crear** un servicio (categoría, título, descripción y precio).
- **Editar** un servicio existente.
- **Archivar** un servicio (dejar de mostrarlo).
- **Restaurar** un servicio archivado.
- **Importar** los servicios de ejemplo de una sola vez (botón "Importar servicios actuales").

### Borrado lógico (concepto clave)

Cuando se "elimina" un servicio, en realidad **no se borra** de la base de datos: solo se marca como archivado ( `active = false` ) y deja de mostrarse. Esto se hace así para **no perder información** que los reportes puedan necesitar en el futuro.



## 12. Módulo 4 — Reservas

**Archivos** parte pública `src/app/reservar/page.tsx` · panel `src/app/admin/reservations/page.tsx` · lógica `src/lib/booking.ts`

### Del lado de la clienta

1. Elige un servicio, una fecha y una hora disponible.
2. El sistema muestra las horas según el **horario del salón** (definido en `site.ts`): Lunes a Viernes de 8:00 a 18:00, Sábado de 8:00 a 16:00, Domingo cerrado.
3. Las horas que ya están llenas aparecen **tachadas** y no se pueden elegir.
4. Al confirmar, la reserva queda como **"pendiente"**.

### Control de cupos (cómo se evita la doble reserva)

Cada hora tiene una **capacidad** (por defecto **2 personas a la vez**, configurable en `site.ts`). El sistema lleva un contador por hora en la colección `slots`:

- 1) Revisa cuántos cupos hay ocupados en esa hora
- 2) Si ya está llena, avisa que no hay disponibilidad
- 3) Guarda la reserva
- 4) Suma 1 al contador de esa hora

### Del lado del administrador

El salón ve las reservas por día y puede cambiar su estado:

| Estado     | Significado                                        |
|------------|----------------------------------------------------|
| Pendiente  | Recién llegó, falta confirmarla.                   |
| Confirmada | El salón la aceptó.                                |
| Completada | El servicio ya se realizó.                         |
| Cancelada  | Se anuló (esto <b>libera el cupo</b> de esa hora). |

## 13. Módulo 5 — Registro diario

---

**Archivos** `src/app/admin/records/page.tsx` · `src/lib/records.ts`

Es la **bitácora** del día: el salón anota cada servicio que realiza.

### Cómo funciona

1. Se elige la fecha, el servicio, la cantidad y (opcional) una nota.
2. Al guardar, el sistema anota una copia del servicio con su precio de ese momento.
3. La pantalla muestra el **total de servicios** y el **total en colones** del día.

**Por qué guarda una copia del precio:** así, si mañana se cambia el precio de un servicio, los registros de días anteriores **siguen mostrando el precio real** que se cobró ese día. Esto hace que los reportes sean confiables.

## 14. Módulo 6 — Reportes

---

**Archivos** `src/app/admin/reports/page.tsx`

Muestra un resumen de lo que se registró en el módulo anterior.

### Qué muestra

- **Selector de período:** Hoy, Últimos 7 días, Este mes, Mes anterior o un rango personalizado.
- **Ingresos totales** del período.
- **Cantidad de servicios realizados.**
- **Ingresos por categoría:** una barra por cada categoría con su monto y porcentaje.

### Cómo hace los cálculos

El sistema pide a la base de datos todos los registros del período elegido y los suma con operaciones simples (recorre la lista, va sumando los ingresos y agrupando por categoría). No hay nada guardado "pre-calculado": los números se arman en el momento con la información real.

## 15. Flujo completo del sistema (ejemplo de principio a fin)

---

Para entender cómo se conecta **todo**, sigamos el recorrido de una cita real, desde que la clienta reserva hasta que aparece en los reportes:

- 1 El salón carga sus servicios.** Desde el módulo de Servicios, el admin crea (o importa) los servicios con sus precios. Se guardan en la colección `services`.
- 2 Los servicios aparecen en la web.** La página pública y el formulario de reservas leen la colección `services` y muestran los servicios activos.
- 3 La clienta reserva.** Entra a "Reservar", elige "Corte & Peinado" para el viernes a las 14:00, deja su nombre y WhatsApp. Se crea un documento en `reservations` con estado **pendiente** y se suma 1 al contador de esa hora en `slots`.
- 4 El admin ve la reserva.** En el módulo de Reservas aparece la cita del viernes como "pendiente". El admin la **confirma**.
- 5 El día de la cita, se completa.** Cuando la clienta llega y se atiende, el admin marca la reserva como **completada**.
- 6 Se anota en el registro diario.** El admin registra el servicio realizado ese día en el módulo de Registro diario. Se guarda en `records` con una copia del precio.
- 7 Aparece en los reportes.** En el módulo de Reportes, ese servicio ya suma a los ingresos del día, de la semana y del mes, y al total de la categoría "Cabello".

**La idea central:** cada módulo aporta datos que otro módulo aprovecha. Los servicios alimentan las reservas y el registro; el registro alimenta los reportes. Todo conectado a través de la base de datos.

## 16. Seguridad (reglas de la base de datos)

Archivo `firestore.rules`

Las **reglas** definen quién puede leer y escribir cada colección. Se configuran en la consola de Firebase. Este es el resumen:

| Colección    | Quién puede leer                          | Quién puede escribir                          |
|--------------|-------------------------------------------|-----------------------------------------------|
| services     | Cualquiera (la web muestra los servicios) | Solo administradores con sesión               |
| records      | Solo administradores                      | Solo administradores                          |
| reservations | Solo administradores                      | Las clientas pueden crear; editar solo admins |
| slots        | Cualquiera (solo es un número de cupos)   | Cualquiera (al reservar)                      |

Los datos privados de las clientas (nombre, teléfono) **no se muestran** en la parte pública: solo los administradores los pueden ver.

## 17. Preguntas típicas de la exposición (con respuestas)

---

Estas son preguntas que suelen hacer al presentar un proyecto así. Estudié las respuestas para defenderlo con seguridad.

### 1. ¿Por qué elegiste Next.js y React?

Porque permiten construir páginas modernas y rápidas dividiendo la interfaz en componentes reutilizables. Son de las herramientas más usadas hoy, gratuitas y con mucha documentación.

### 2. ¿Por qué usaste Firebase y no una base de datos como MySQL?

Firebase incluye la base de datos y el inicio de sesión ya listos, sin tener que programar ni pagar un servidor propio. Para un proyecto de este tamaño simplifica mucho el trabajo y funciona en la nube.

### 3. ¿Qué es un componente?

Una pieza reutilizable de la interfaz, como un botón o una tarjeta. Se programa una vez y se usa en varios lugares. Por ejemplo, el botón de WhatsApp es un componente que aparece en varias secciones.

### 4. ¿Cómo funciona el inicio de sesión? ¿Dónde se guardan las contraseñas?

El login usa Firebase Authentication. Las contraseñas no están en mi código ni en mi base de datos: las guarda y verifica Firebase de forma segura. El sistema solo pregunta "¿este correo y contraseña son válidos?".

### 5. ¿Qué es el "borrado lógico" y por qué lo usaste?

En lugar de borrar un servicio de verdad, lo marco como archivado (un campo `active` en falso). Así deja de mostrarse pero no se pierde la información, que puede ser necesaria para los reportes históricos.

### 6. ¿Cómo evitas que dos clientas reserven el mismo cupo?

Cada hora tiene una capacidad y un contador en la colección `slots`. Antes de guardar una reserva, el sistema revisa cuántos cupos hay ocupados; si ya está lleno, no permite reservar. Al reservar, suma 1 al contador.

### 7. ¿Por qué guardas una copia del precio en las reservas y registros?

Para que los reportes históricos sean correctos. Si más adelante cambio el precio de un servicio, los registros viejos siguen mostrando el precio real que se cobró ese día.

### **8. ¿Qué son las reglas de Firestore?**

Son las que definen quién puede leer y escribir cada colección. Por ejemplo, cualquiera puede ver los servicios, pero solo un administrador con sesión puede modificarlos, y los datos de las clientas solo los ve el admin.

### **9. ¿Las clientas necesitan crear una cuenta para reservar?**

No. Reservan como invitadas, solo con su nombre y WhatsApp. Esto reduce la fricción y aumenta la probabilidad de que reserven.

### **10. ¿Cómo se conectan los módulos entre sí?**

A través de la base de datos. Los servicios alimentan las reservas y el registro diario; el registro diario alimenta los reportes. (Ver el capítulo 15, "Flujo completo del sistema".)

### **11. ¿Qué mejorarías si tuvieras más tiempo?**

Agregar notificaciones automáticas al salón cuando llega una reserva, protección anti-spam en el formulario (reCAPTCHA), y publicar el sitio en internet para uso real.

### **12. ¿La información se pierde si apago la computadora?**

No. Los datos están en la nube (Firebase), no en la computadora. Apagar el programa solo detiene la página; la información sigue guardada.

## 18. Problemas comunes y soluciones

| Problema                                                     | Solución                                                                                                                       |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Al correr <code>npm run dev</code> dice "command not found". | No está instalado Node.js. Repetí el <b>Paso 1</b> del capítulo 4.                                                             |
| La página dice "No se pudieron cargar los servicios".        | La base de datos (Firestore) no está creada, o se acaba de crear y tarda unos segundos. Esperá un momento y recargá.           |
| El inicio de sesión no funciona.                             | Verificá que el usuario exista en Firebase (Authentication → Users) y que el método "Correo/contraseña" esté activado.         |
| Los servicios aparecen vacíos.                               | Entrá al módulo de Servicios y usá el botón "Importar servicios actuales".                                                     |
| La página no abre en el navegador.                           | Revisá que la Terminal muestre "Ready" y que estés entrando a <code>http://localhost:3000</code> .                             |
| Cambié el horario o la capacidad y no se refleja.            | Esos valores están en <code>src/content/site.ts</code> (sección <code>booking</code> ). Guardá el archivo y recargá la página. |

**Fin de la documentación**  
Sistema Web — Gaby Salón y Spa